

Day 3: Data Quality & Test Engineering Report

1. Scope & System Architecture

Day 3 focused on establishing the defensive engineering layer: constructing automated data quality contracts, unit/integration testing suites, and Continuous Integration (CI) pipelines to intercept and catch data corruption prior to model training.

AI Disclosure Statement: Large Language Model (LLM) tooling was utilized as an automated pair programmer across the project lifecycle for test suite construction, CI workflow configuration, and technical report drafting.

2. Ingestion Contracts & Schema Verification

Machine learning pipelines in production are highly vulnerable to silent failures. Schema contracts were enforced at ingestion to prevent downstream failures:

- **Type Integrity:** Ensures continuous numeric attributes cannot ingest unparsed string variables without immediate type validation and coercion.
- **Missing Value Upper Bounds:** Triggers alerts if missingness exceeds 5% on critical predictive attributes.
- **Category Drift Handling:** Unseen categorical levels in incoming production batches are handled gracefully via `handle_unknown='ignore'` rather than throwing unhandled runtime exceptions.
- **Primary Key Non-Uniqueness:** Enforces structural group boundaries across identifiers.

3. Automated Test Suite Matrix

The following unit and integration tests were implemented and verified within the repository test suite:

Test Module	Test Description	Failure Criterion	Status
<code>test_schema_integrity</code>	Validate column presence, names, and explicit data types	Missing/mismatched column name	PASSED

Test Module	Test Description	Failure Criterion	Status
test_missingness_burst	Ensure feature missing rate remains below 5%	Any single feature null rate > 5%	PASSED
test_pipeline_determinism	Repeated transformations on identical batches must yield bit-identical arrays	Difference in transformed output > 0	PASSED
test_target_leakage_guard	Pre-training assertion checking that baseline AUC remains below 0.98	Baseline AUC $\geq$ 0.98 flags potential leakage	PASSED

4. Continuous Integration (CI/CD) Workflow

The full test suite was integrated into GitHub Actions continuous integration workflows:

- **Automated Guardrails:** Pull requests modifying data preprocessors or introducing schema-breaking transformations are automatically blocked prior to merge, ensuring production deployment stability.
- **CI Run Verification (Pass/Fail):**
 - *Failed Run:* Triggered on an intentionally corrupted schema / unpruned leakage dataset where the leakage assertion caught AUC = 1.00 and failed the build.
 - *Passed Run:* Executed on commit [8f4a1c2](#) with pruned features and clean schema validation, completing all checks with exit code 0.
- **Environment Locking:** All package dependencies are pinned with exact versions to ensure bit-for-bit identical execution across CI runners and local developer environments.